

ArgMax Mini

Kubernetes-based MLOps Platform - System Architecture & Design

Executive Summary

ArgMax Mini는 최대 2GB의 CSV/XLSX를 받아 정형 데이터 모델을 학습시키고, 검증된 모델을 배포해 동기 추론을 제공하는 플랫폼이다. 설계의 중심은 대용량 전송과 12-24시간 GPU 학습을 HTTP 요청 수명에서 분리하는 데 있다. API는 요청 검증과 업무 상태 변경만 담당하고, 실제 파일 처리와 학습은 Kubernetes Job에서 수행한다.

RDS PostgreSQL은 업무 상태(business state)와 실행 의도(execution intent)를 관리하는 단일 신뢰 원천(SSOT)이다. S3는 원본 데이터, 처리된 Parquet, checkpoint, 변경 불가능한 모델 아티팩트를 저장한다. SQS는 상태 저장소가 아니라 Controller 실행을 유도하는 at-least-once wake-up signal이다. 따라서 메시지 중복, 유실, 지연, Controller 재시작, Kubernetes 관측 실패가 발생해도 각 이벤트 소비자(Consumer)는 RDS의 최신 상태를 다시 읽고 멍등적으로 동작한다. 결과가 불확실한 외부 호출은 무조건 재시도하지 않고 Reconciliation으로 실제 상태를 확인한다.

운영 목표는 Amazon EKS 기반이다. GPU 학습은 Karpenter가 공급하는 GPU NodePool에서 Kubernetes Job으로 실행하며, 서빙은 CPU XGBoost 런타임을 사용하는 KServe Standard Mode로 분리한다. 외부 추론 요청은 Inference Gateway만 통과하고 KServe는 cluster-local 경계를 유지한다. 초기 기준 구성은 XGBoost classifier/regressor와 model.json 아티팩트에 한정한다.

이 문서는 운영 목표 설계와 과제 구현 범위를 구분한다. AWS/EKS, Controller, GPU 학습, KServe, MLflow, Athena는 운영 설계이며, 실제 과제 구현은 PostgreSQL 기반 데이터 모델과 제한된 Dataset 중심 API, Docker Compose 로컬 평가 환경까지다.

1. Requirements & Design Strategy

1.1 Problem and scope

핵심 요구사항은 파일 크기, 실행 시간, 모델 계보(provenance), 운영 경계가 서로 다른 네 가지 수명 주기를 동시에 다룬다는 데 있다. 단일 API 서버가 파일을 중계하거나 학습을 직접 실행하면 확장성, 장애 격리, 재시작 가능성이 모두 약해진다. 따라서 Control Plane은 RDS 상태와 오케스트레이션 의도를 관리하고, Data Plane은 S3 객체와 Kubernetes 워크로드를 처리한다.

Requirement	Design challenge
CSV/XLSX 최대 2GB	API proxy 없이 안전한 직접 업로드
12-24h GPU 학습	요청 수명 주기와 실행 수명 주기 분리
모델 버전 관리	변경 불가능한 아티팩트와 추적 가능한 계보
배포와 동기 추론	비즈니스 경계와 서빙 런타임 분리
비동기 작업	DB, 메시지, Kubernetes 상태 불일치 복구
분석	OLTP와 분석 워크로드 분리

1.2 Design principles

- RDS PostgreSQL은 업무 상태와 실행 의도의 SSOT다.
- 장기 작업은 동기 API 요청에서 분리한다.
- SQS는 Controller 실행을 유도하는 at-least-once wake-up signal이며 작업 상태의 SSOT가 아니다.
- 중복 전달을 전제로 Consumer 멍등성을 설계한다.
- 불확실한 외부 결과는 Reconciliation 후 RDS SSOT에 기록된 상태로 수렴시킨다.
- DatasetVersion과 ModelVersion 아티팩트는 변경 불가능하게 유지한다.
- 외부 API 경계와 내부 모델 실행 경계를 분리한다.
- 현재 요구에 기여하지 않는 운영 복잡도는 의도적으로 제외한다.

2. Overall System Architecture

2.1 Boundary and flow

외부 요청은 Route 53, WAF, internet-facing ALB를 거쳐 Backend API 또는 Inference Gateway에만 도달한다. API, Outbox Publisher, Controller, Job, Gateway, KServe, MLflow, Prometheus/Grafana는 Private Application Subnet의 EKS에서 실행된다. RDS PostgreSQL은 Private Data Subnet에 Publicly Accessible=false로 배치하고, S3, SQS, Glue, Athena, CloudWatch는 AWS 리전 서비스로 사용한다.

Private EKS 워크로드의 S3 접근은 S3 Gateway Endpoint를 사용한다. 그 밖의 외부 트래픽은 AZ별 NAT Gateway를 통해 나가며, Private Data Subnet에는 인터넷 기본 경로가 없다. KServe와 MLflow는 공개 대상이 아니다. KServe는 Gateway가 호출하는 cluster-local ClusterIP Service다. MLflow는 Private Application 영역에 배치하고, ML Engineer가 승인된 VPN 또는 조직 내부 접근 경로에서 인증한 뒤에만 접근한다. 특정 연결 제품이나 새로운 인증 시스템은 이 설계에서 추가하지 않는다.

2.2 Control plane and data plane

Control Plane은 API와 Controller가 RDS 행과 OutboxEvent를 변경하고 실제 상태를 관찰하는 영역이다. Data Plane은 S3의 대용량 변경 불가능 객체, GPU/CPU Pod 실행, KServe 추론 요청을 처리한다. Kubernetes의 Pod 상태나 SQS 메시지 존재 여부는 관측값일 뿐 업무 상태의 SSOT가 아니다.

Figure 1 | Overall Architecture

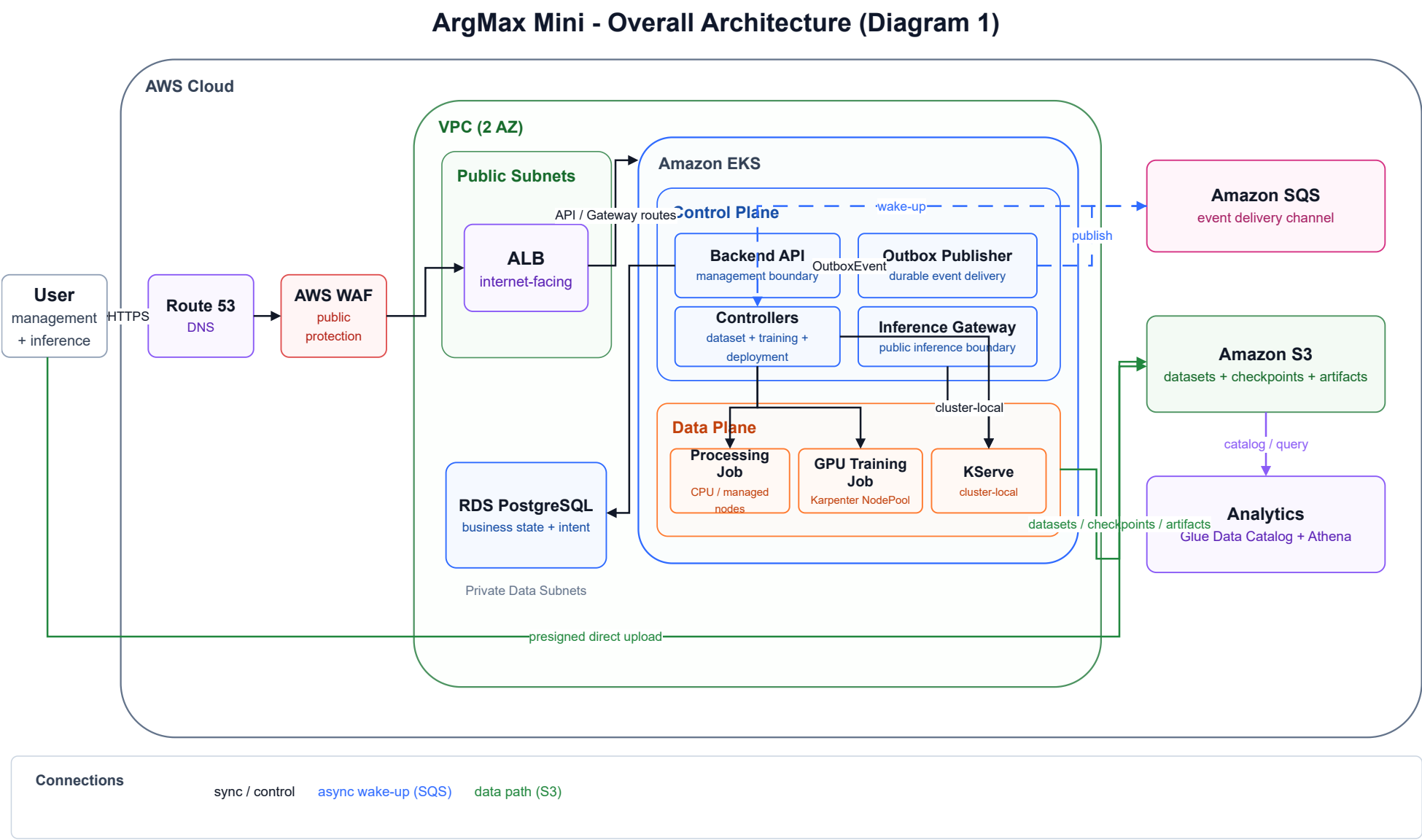
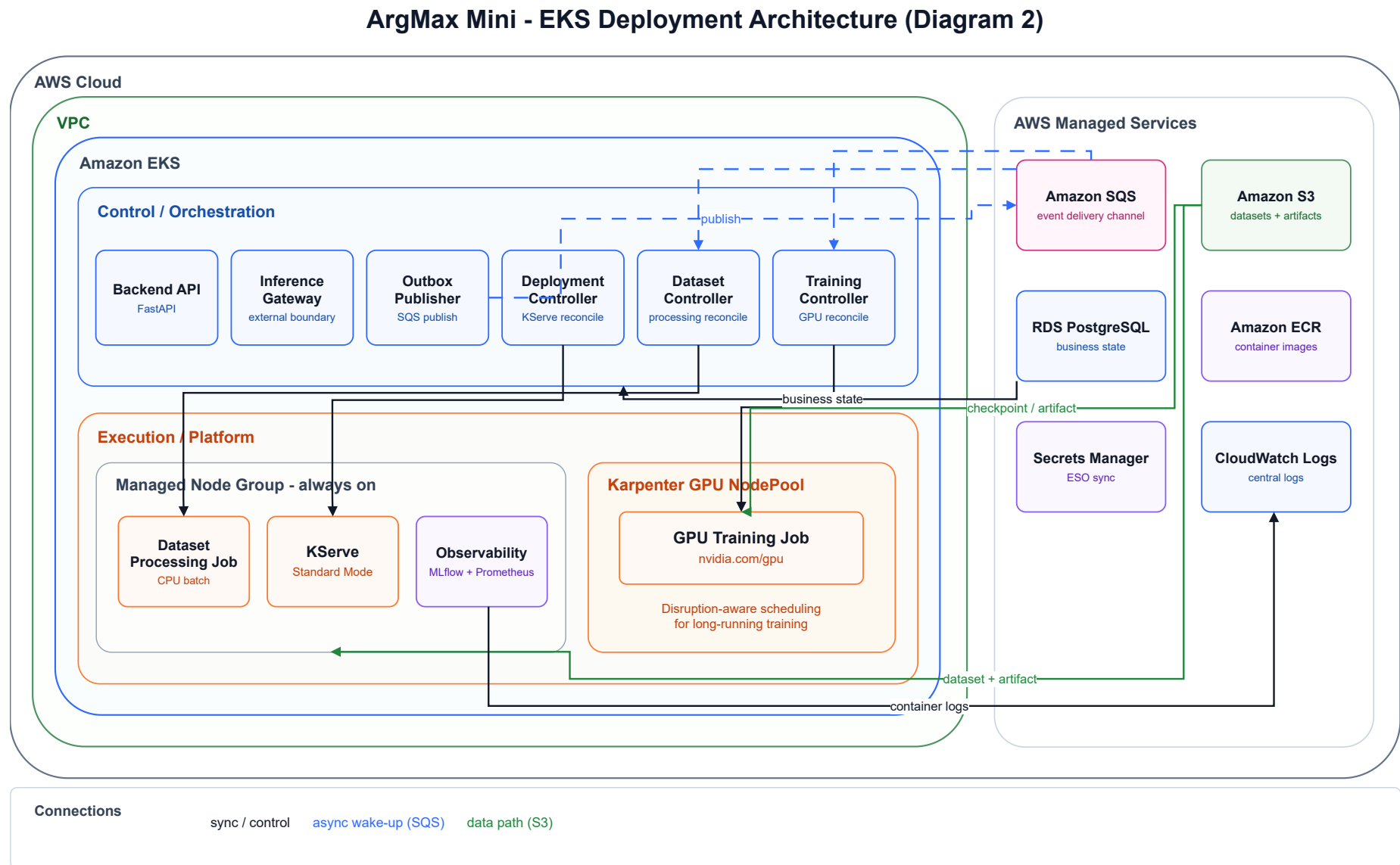


Figure 1. Overall System Architecture. 외부 요청 흐름, AWS 서비스 경계, EKS Control/Data Plane 분리, RDS-S3-SQS 책임 경계를 표현한다.

Figure 2. EKS Deployment Architecture. Control Plane의 오케스트레이션 책임과 Execution Plane의 워크로드 실행 경계를 표현한다.



2.3 Technology stack

Area	Choice	Why
API	FastAPI + Pydantic v2	Python ML 생태계와 명시적 검증
Persistence	SQLAlchemy + Alembic	PostgreSQL 트랜잭션과 스키마 마이그레이션
Container platform	Amazon EKS	Job, GPU 스케줄링, KServe 통합
OLTP	RDS PostgreSQL	FK, 트랜잭션, 업무 상태
Object storage	Amazon S3	대용량 변경 불가능 객체
Async	Amazon SQS	관리형 at-least-once 이벤트 전달
GPU compute	Kubernetes Job + Karpenter	장기 GPU 워크로드 격리
Serving	KServe Standard Mode	실행 수명 주기 분리
Experiment	MLflow	Run, metric, 환경 추적
Analytics	S3 + Glue + Athena	OLTP와 분석 워크로드 분리
Observability	Prometheus/Grafana + CloudWatch	Kubernetes와 AWS 계층 관측
Secrets	Secrets Manager + ESO	장기 자격 증명 제거
Delivery	GitHub Actions + Helm + Argo CD	빌드와 desired state 분리

Design Decision - EKS as the runtime platform

Problem: 장기 GPU Job, KServe, autoscaling을 한 운영 경계에서 다뤄야 한다.

Choice: Amazon EKS를 운영 런타임으로 사용한다.

Why: Kubernetes Job, Karpenter GPU NodePool, KServe를 같은 스케줄러와 정책 모델로 통합한다.

Alternative: ECS/Fargate/Lambda.

Trade-off: 클러스터 운영 복잡도와 Controller 관측 책임이 추가된다.

Design Decision - RDS SSOT

Problem: DB, SQS, Kubernetes가 서로 다른 시점의 상태를 보일 수 있다.

Choice: RDS에 업무 상태와 실행 의도를 기록한다.

Why: 트랜잭션, FK, 조건부 갱신으로 수명 주기와 계보를 보호한다.

Alternative: Queue 또는 Kubernetes 상태를 기준으로 사용.

Trade-off: Controller가 관측 결과를 RDS SSOT에 기록된 상태로 수렴시키는 책임을 갖는다.

3. Dataset Upload & Processing

3.1 2GB 직접 업로드

2GB 파일을 Backend API가 중계하면 대역폭, 메모리·스토리지 압박, 시간 초과, 수평 확장 효율이 악화된다.

Client는 먼저 API에서 DatasetVersion과 UploadSession, presigned URL을 받고 S3로 직접 업로드한다. ≤ 16 MiB는 SINGLE_PUT, 초과 파일은 MULTIPART이며 최대 크기는 2GB다.

업로드 완료 API는 객체 존재 여부와 ContentLength를 확인한다. Multipart의 HeadObject.ChecksumSHA256를 전체 파일 SHA-256과 직접 비교하지 않는다. Client가 제시한 전체 파일 SHA-256의 독립 검증은 Processing Job이 원본 객체를 스트리밍 방식으로 읽으면서 수행한다. 따라서 UPLOADED는 객체와 크기 검증이 끝난 상태이지 서버가 전체 파일 해시를 독립 검증한 상태를 뜻하지 않는다.

완료 성공 시 하나의 PostgreSQL 트랜잭션에서 UploadSession을 COMPLETED로 전이하고 DatasetVersion 메타데이터를 기록하며 DatasetVersion을 UPLOADED로 바꾸고 OutboxEvent를 삽입한다. PostgreSQL 트랜잭션 커밋 뒤 Publisher가 SQS 메시지를 발행한다. 이 순서는 DB 상태 변경과 이벤트 의도의 원자성을 보장한다.

Design Decision - S3 direct upload

Problem: 대용량 파일을 API와 같은 실패/확장 경계에 둘 수 없다.

Choice: presigned S3 직접 업로드를 사용한다.

Why: API는 인가와 메타데이터만 담당하고 데이터 전송을 분리한다.

Alternative: API 중계 업로드.

Trade-off: Multipart 완료, 만료, 객체 Reconciliation을 명시적으로 다뤄야 한다.

3.2 Processing and idempotency

Dataset Processing Controller는 SQS 메시지를 받은 뒤 DatasetVersion ID로 RDS의 최신 상태를 읽고, 처리 대상일 때만 Kubernetes Processing Job을 만든다. Job은 원본 읽기, 전체 파일 checksum 검증, CSV/XLSX 파싱, Parquet 변환, DatasetColumn 스키마-통계 생성, 처리 결과의 S3 저장을 수행한 뒤 READY 또는 FAILED 결과를 기록한다. Controller는 오케스트레이션과 Reconciliation을, Job은 대용량 파일 처리를 담당한다.

기본 수명 주기는 PENDING -> UPLOADING -> UPLOADED -> PROCESSING -> READY다. 업로드 또는 처리 단계의 확정 오류는 FAILED로 전이한다. CompleteMultipartUpload 결과처럼 외부 결과가 불확실하면 즉시 실패로 단정하지 않고 HeadObject 또는 ListParts로 실제 상태를 확인한다. 메시지 중복은 조건부 갱신, 결정적 Job 식별자, 최신 RDS 상태 재조회로 무해하게 만든다.

Design Decision - Controller plus Job

Problem: 오케스트레이션과 대용량 파싱은 서로 다른 실패 특성을 가진다.

Choice: Controller는 Reconciliation, Job은 파일 처리를 맡는다.

Why: Controller 재시작과 대용량 처리 실패를 독립적으로 복구한다.

Trade-off: RDS 상태와 Kubernetes 실제 상태를 지속적으로 대조해야 한다.

Figure 3 | Dataset Processing Flow

ArgMax Mini - Dataset Upload and Processing Flow (Diagram 3)

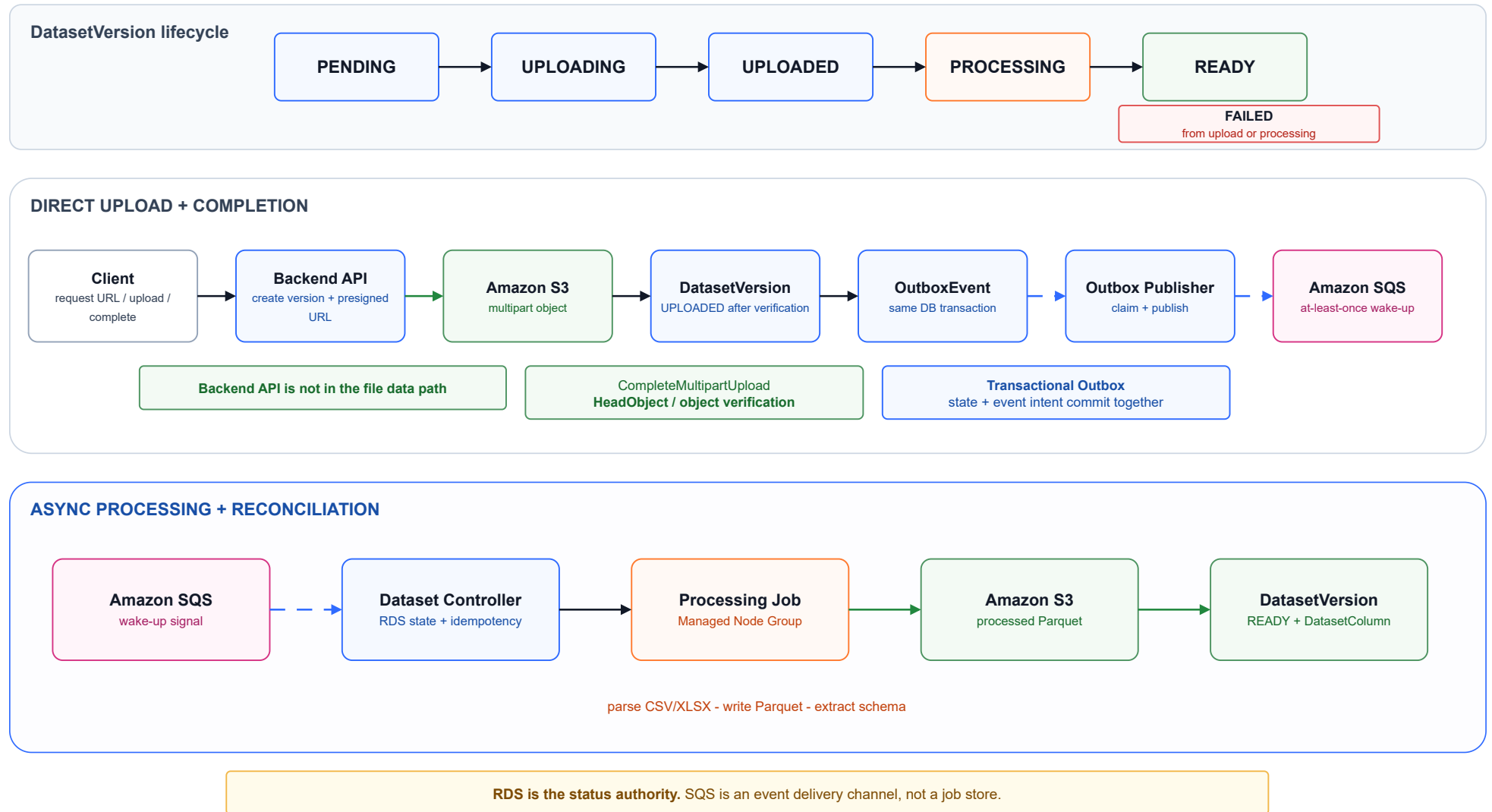


Figure 3. Dataset Upload / Processing. Client의 직접 업로드 완료 후 DB transaction에서 OutboxEvent를 기록하고, Controller가 Processing Job을 생성하는 비동기 처리 흐름을 표현한다.

4. Long-running GPU Training & Recovery

4.1 Lifecycle, runtime, memory and checkpoint

12-24시간 GPU 작업을 API Pod나 일반 요청 수명 주기에 묶으면 취소, 재시도, 자원 스케줄링, 장애 격리가 어렵다. API는 TrainingJob을 QUEUED로 만들고 OutboxEvent를 같은 트랜잭션에 기록한다. Outbox Publisher가 SQS 메시지를 발행하면 Training Controller는 최신 TrainingJob과 quota를 RDS에서 재조회하고 Kubernetes GPU Job을 생성한다.

기본 상태는 QUEUED -> SCHEDULING -> RUNNING -> SUCCEEDED / FAILED이며, 취소는 CANCEL_REQUESTED -> CANCELLED로 수렴한다. 상태 변경은 예상 현재 상태를 조건으로 둔 조건부 갱신으로 보호한다. Controller는 고아 Job, 시간 초과, 정제된 스케줄링을 관찰하고 RDS SSOT에 기록된 상태를 기준으로 복구한다.

상시 워크로드는 Managed Node Group에서 실행하고 GPU Training Job은 Karpenter GPU NodePool에 배치한다. On-Demand를 기준으로 하며 GPU taint/label과 training toleration/affinity로 분리한다. 사용자 quota와 NodePool GPU limit은 스케줄링 직전에 확인하며, 유류 GPU 용량은 0까지 축소할 수 있다. Spot은 checkpoint와 재시도 검증 이후의 향후 검토 대상이다.

초기 Algorithm Registry는 XGB00ST_CLASSIFIER, XGB00ST_REGRESSOR만 지원한다. Training runtime은 tree_method=hist, device=cuda를 사용하고 CPU override를 허용하지 않는다. Pod는 nvidia.com/gpu request와 limit을 동일하게 설정한다. 처리된 Parquet는 XGBoost의 메모리 효율적인 quantized input 경로로 읽는다.

GPU VRAM 사용량을 사전에 정확히 예측한다고 주장하지 않는다. Dataset profile과 GPU VRAM은 사전 위험 신호이며, 실제 메모리 압박이 확인되면 XGBoost external-memory 경로를 확장안으로 검토한다. RAPIDS, RMM, Dask-CUDA, multi-GPU distributed XGBoost는 초기 기준 구성에 넣지 않는다.

Job은 주기적으로 checkpoint를 S3에 저장하고 메타데이터를 RDS에 기록한다. 재시도 Pod는 최신 유효 checkpoint에서 재개하며 기존 TrainingJob deadline과 MLflow parent Run을 공유한다. Job Complete만으로 TrainingJob 성공을 선언하지 않는다. Controller는 정상 종료, READY ModelVersion 존재, CREATING 후보 소멸, 아티팩트와 ModelInterface의 finalization을 확인한 후 SUCCEEDED로 전이한다.

Figure 4 | Training Pipeline

ArgMax Mini - Training Pipeline and Model Publishing (Diagram 4)

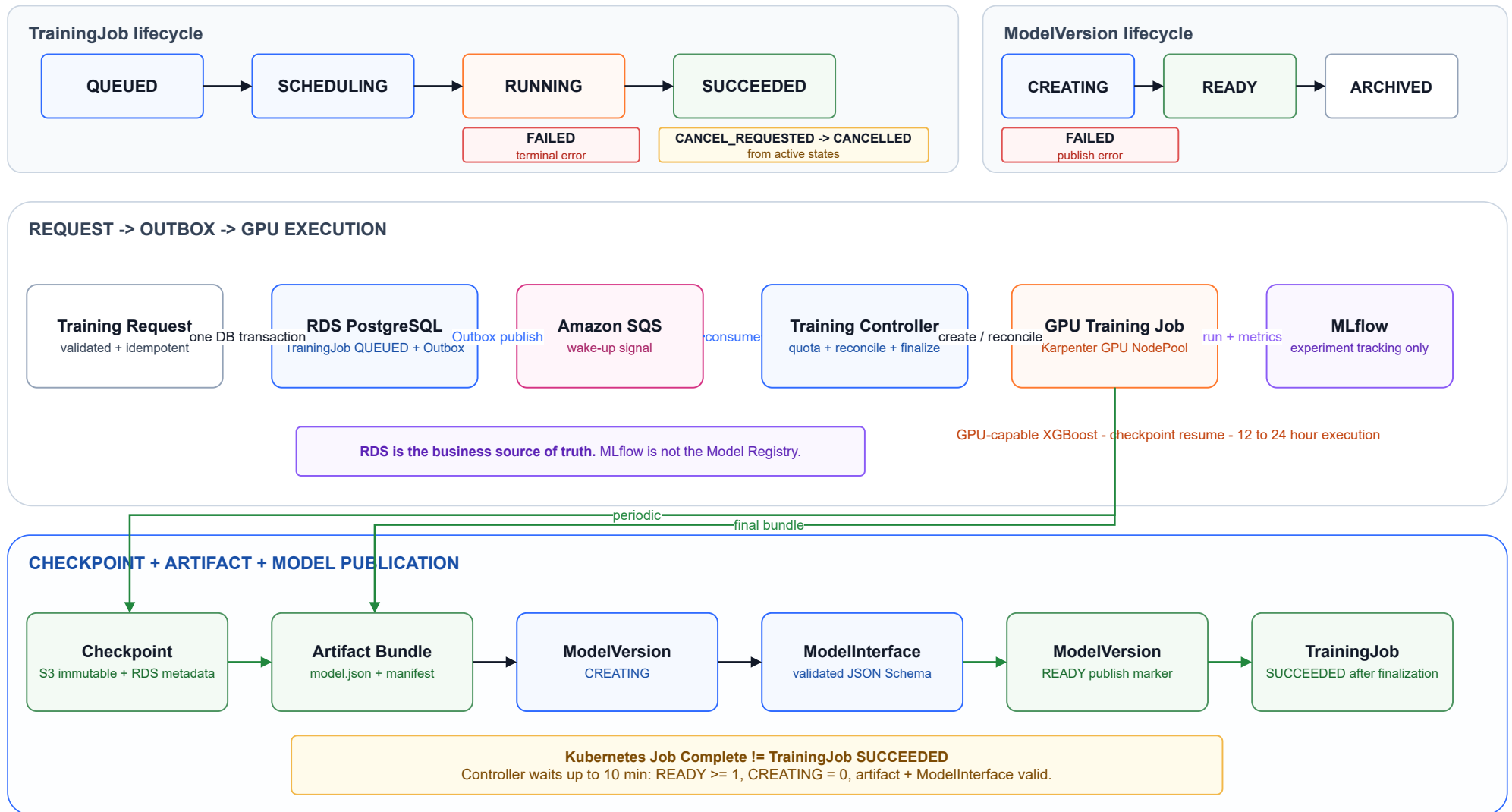


Figure 4. Training / Checkpoint / Model Publish. GPU Job의 checkpoint 저장, 재시작 복구, immutable ModelVersion 발행과 Controller finalization 과정을 표현한다.

4.2 Outbox, reconciliation and recovery

DB 변경과 OutboxEvent 기록은 하나의 트랜잭션 커밋으로 함께 확정한다. Publisher는 OutboxEvent를 SQS에 전달하고, Consumer는 중복 전달을 허용한다. 메시지 누락, DLQ 이동, Controller 장애는 주기적 Reconciler가 RDS의 대기 중인 실행 의도와 Kubernetes 실제 상태를 비교해 복구한다. 즉, SQS 메시지가 사라져도 RDS는 복구의 SSOT로 남고 Reconciler는 이를 기준으로 복구한다.

GPU 중단, Node 손실, 일시적인 네트워크·S3·RDS 실패는 checkpoint 기반 재시도 후보가 된다. 잘못된 algorithm/hyperparameter, Dataset 불일치, 결정적 코드 오류, CUDA/host OOM, 아티팩트·인터페이스 생성 실패는 재시도하지 않는 기본 오류다. 취소 요청은 Job 삭제 전에 실제 Complete/Failed/running 상태를 관찰해 이미 검증된 결과를 보존하거나 안전하게 종료한다.

Design Decision - GPU Job plus Karpenter

Problem: 장기 GPU 실행은 API와 상시 Pod 용량에서 분리되어야 한다.

Choice: Kubernetes GPU Job과 Karpenter NodePool을 사용한다.

Why: 스케줄링, 수명 주기, checkpoint 복구, 유휴 용량 축소를 함께 다룬다.

Alternative: 항상 커진 GPU fleet 또는 serverless execution.

Trade-off: 스케줄링 지연과 Controller Reconciliation 책임이 생긴다.

Design Decision - SQS plus Transactional Outbox

Problem: DB 트랜잭션 커밋과 메시지 발행 사이의 부분 실패를 피해야 한다.

Choice: 도메인 변경과 OutboxEvent를 하나의 트랜잭션 커밋으로 확정된 뒤 비동기로 발행한다.

Why: 지속 가능한 실행 의도와 Consumer 멍등성을 결합한다.

Trade-off: Publisher와 Reconciliation 절차가 필요하다.

5. Model Versioning, Deployment & Inference

5.1 Model version, deployment and serving boundary

Model은 논리적 사용자 리소스이고 ModelVersion은 특정 TrainingJob이 만든 변경 불가능한 학습 결과다. ModelVersion은 DatasetVersion, algorithm, 학습 설정, metric, 실행 환경, 아티팩트 메타데이터와 연결되어 계보(provenance)를 보존한다. 배포는 하나의 변경 불가능한 ModelVersion에 고정된다. 새 버전을 사용하려면 새 InferenceDeployment를 만든다. in-place model 교체, stable alias, canary/traffic split, 자동 승격은 초기 범위에서 제외한다.

초기 아티팩트 묶음은 manifest.json, interface.json, preprocessing.json, classifier의 labels.json, XGBoost model.json으로 구성한다. 임의 Python 코드, dynamic import, pickle, joblib는 허용하지 않는다. 전처리는 선언형 변환만 사용한다. READY는 검증된 아티팩트와 ModelInterface가 모두 확정되었음을 나타내는 발행 표식이다.

ModelInterface는 단일 인스턴스의 input/output JSON Schema와 feature order를 보관한다. Gateway가 사용자 요청을 검증할 수 있게 하고, 모델 런타임의 내부 형식이 공개 API 계약을 침범하지 않게 한다. ModelInterface는 ModelVersion과 함께 변경 불가능하게 관리한다. 호환되지 않는 입출력 스키마 변경은 기존 인터페이스를 수정하지 않고 새로운 ModelVersion을 발행해 처리하며, 기존 InferenceDeployment는 배포 시점의 interface contract를 유지한다.

Deployment Controller는 RDS의 PENDING 배포를 감지해 변경 불가능한 아티팩트, manifest, 런타임, interface, checksum을 확인하고 KServe InferenceService를 생성한다. KServe readiness가 확인된 뒤 InferenceDeployment를 READY로 전이한다. Controller는 watch 기반 빠른 경로와 주기적 Reconciliation을 함께 사용해 RDS와 KServe의 상태 차이를 복구한다.

Inference Gateway는 유일한 외부 추론 애플리케이션 경계다. Gateway는 인증, 소유권-상태 확인, 비즈니스 검증, ModelInterface 스키마 검증, 표준 오류 계약을 담당하고, 결정적인 namespace/service name으로 cluster-local KServe ClusterIP Service를 호출한다. KServe는 아티팩트 로드, XGBoost CPU 런타임, 실제 추론 실행만 담당하며 별도 공개 endpoint를 제공하지 않는다.

학습은 GPU/CUDA를 사용하지만 초기 서버는 CPU XGBoost 런타임이다. 이는 지연 시간과 운영 비용이 다른 서버 경로에 GPU를 상시 할당하지 않기 위한 선택이다.

서빙 자동 확장은 초기에는 KServe의 기본 정책을 사용한다. 활성 배포는 min_replicas >= 1을 유지하고, 모델별 요청 동시성(request concurrency)과 지연 시간(latency) 지표를 관찰해 확장 기준을 조정한다. 운영 복잡도를 제한하기 위해 자동 canary 배포와 traffic splitting은 초기 범위에서 제외한다.

Design Decision - Inference Gateway plus KServe

Problem: 외부 API 계약과 모델 실행 수명 주기를 분리해야 한다.

Choice: Gateway 앞단과 cluster-local KServe 런타임을 나눈다.

Why: 인증과 스키마-오류 계약을 모델 런타임 밖에서 일관되게 적용한다.

Trade-off: Gateway 라우팅과 배포 Reconciliation을 운영해야 한다.

Figure 5 | Deployment & Inference

ArgMax Mini - Model Deployment and Inference Flow (Diagram 5)

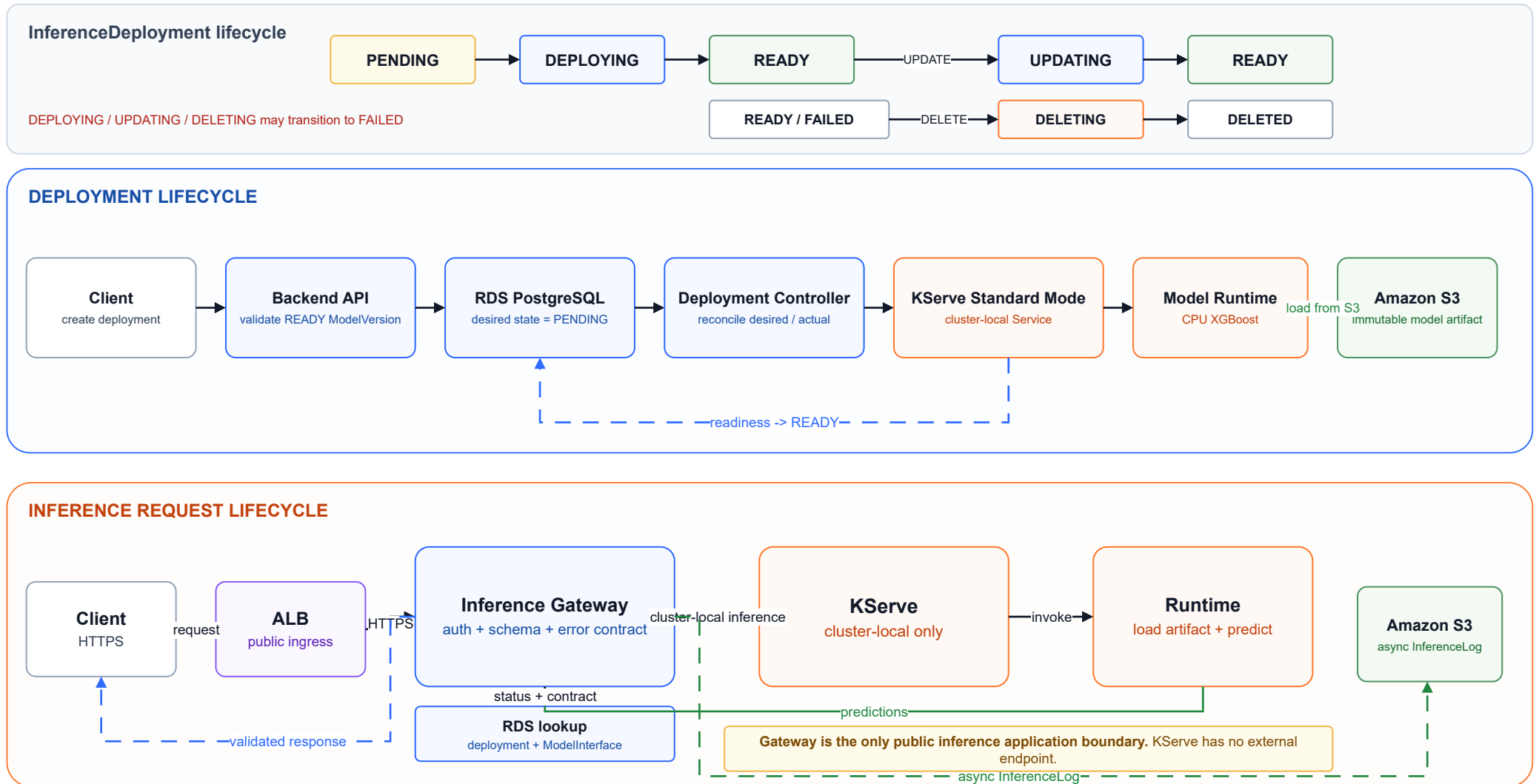


Figure 5. Deployment / Inference. 배포 의도(desired state)가 Controller와 KServe readiness 검증을 거쳐 READY 상태가 되는 과정과 Gateway/KServe serving 경계를 표현한다.

6. Data Model & State Management

6.1 Entity responsibilities

RDB는 14개 핵심 엔티티로 워크플로와 계보(provenance)를 표현한다: User, Dataset, DatasetVersion, DatasetColumn, UploadSession, Model, TrainingJob, TrainingJobEvent, TrainingCheckpoint, ModelVersion, ModelInterface, InferenceDeployment, AuditLog, OutboxEvent. UUIDv4 PK와 TIMESTAMPTZ를 사용하고, 수명 주기 상태는 VARCHAR + CHECK, 관계는 FK/UNIQUE/CHECK로 보호한다.

Dataset과 Model은 논리 리소스이며 soft delete가 가능하다. DatasetVersion과 ModelVersion은 변경 불가능한 물리적 결과다. TrainingJob은 DatasetVersion과 Model을 연결하고, TrainingCheckpoint와 ModelVersion은 이 실행의 계보를 이어간다. InferenceDeployment는 하나의 변경 불가능한 ModelVersion과 연결된다.

OutboxEvent는 트랜잭션으로 확정된 실행 의도를, AuditLog는 관리·업무 변경 감사를 맡는다.

6.2 State ownership

사용자가 status를 직접 PATCH하지 않는다. DatasetVersion은 API와 Processing Job, TrainingJob은 API 취소 요청과 Training Controller, ModelVersion은 Training runtime-Reconciler-archive API, InferenceDeployment는 API와 Deployment Controller가 각각 상태 변경 권한을 갖는다. 중요 갱신은 현재 상태를 조건으로 하는 조건부 갱신을 사용한다. 이 규칙은 중복 전달이나 동시에 실행되는 Controller 관측에서 상태 전이를 보호한다.

Figure 6 | RDB Schema

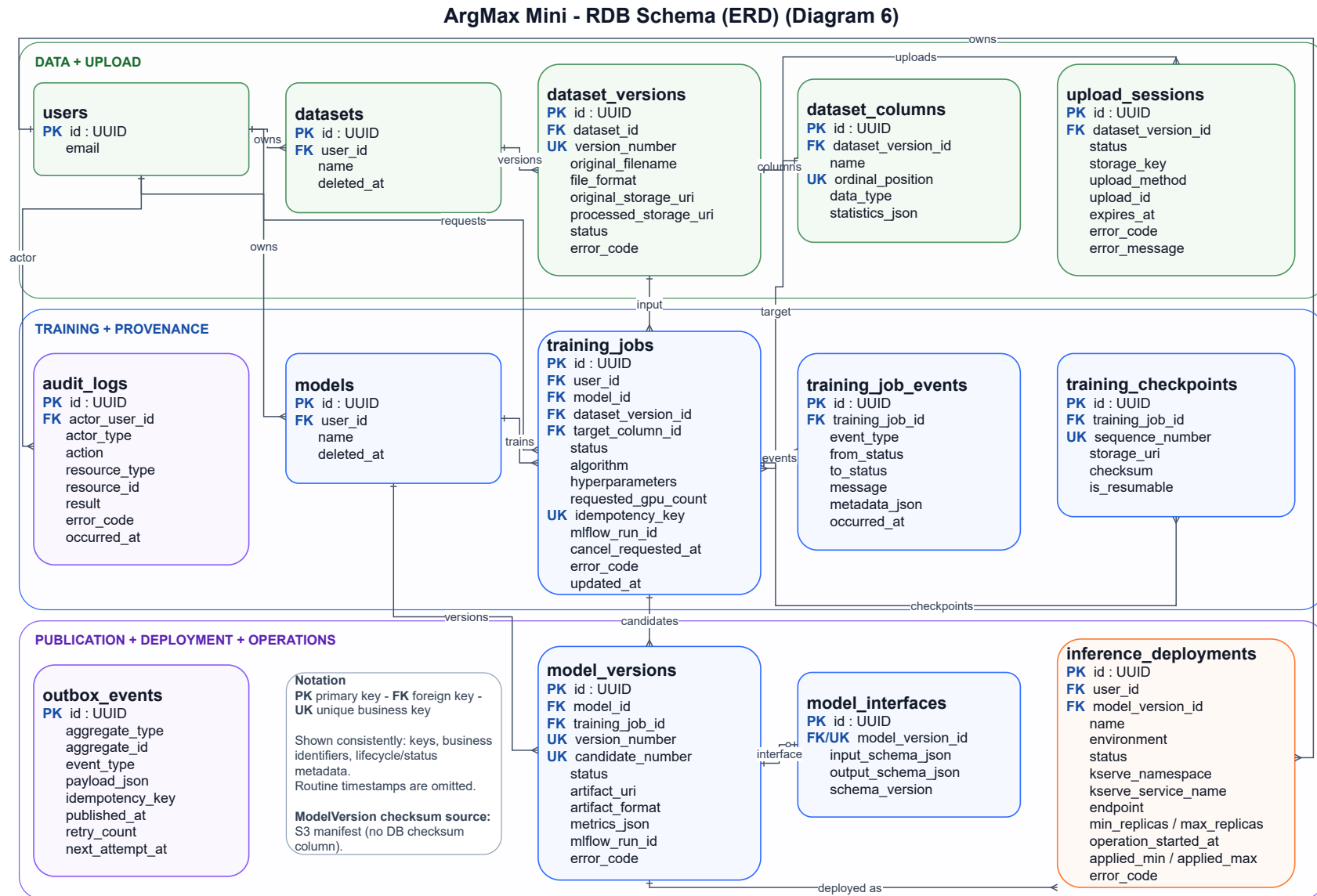


Figure 6. ERD. 논리 리소스와 변경 불가능한 Version, TrainingJob provenance, deployment binding, outbox/audit 책임을 표현한다.

7. Reliability, Security & Observability

7.1 Failure and recovery

공통 복구 원칙은 다음 순서를 따른다: 장애 감지 → 결과 확인 가능 여부 판단 → 실제 상태 조회 → 안전한 범위 내 재시도 → RDS SSOT에 기록된 상태로 수렴. RDS 장애 시에는 변경을 차단(fail closed)하고 오래된 상태로 갱신하지 않는다. 실행 중인 TrainingJob은 연산과 checkpoint를 유지할 수 있지만 DB 복구 후 Reconciliation이 필요하다. RDS Multi-AZ DB instance는 Primary와 synchronous standby로 AZ 수준의 고가용성을 제공한다. Standby는 읽기 확장 용도가 아니며 고가용성은 재해 복구와 같지 않다. DR/RTO/RPO, backup retention, cross-region recovery는 향후 결정 사항이다.

Kubernetes API 또는 Controller 장애는 즉시 FAILED로 단정하지 않고 실제 워크로드를 다시 관찰한다. 단일 추론 오류가 배포 수명 주기를 즉시 FAILED로 바꾸지 않는다. SQS 메시지의 중복·유실·지연은 OutboxEvent와 Reconciliation으로 다룬다.

7.2 Security boundaries

공개 경계에는 Backend API와 Inference Gateway만 둔다. EKS 워크로드, KServe, MLflow, RDS는 Private 영역에 둔다. 인증 공급자 구현을 고정하지 않되 인증된 주체의 user_id를 서버 측 리소스 소유권과 매핑하고, Client가 전달한 사용자 ID는 신뢰하지 않는다. 초기 인가는 소유자 전용이며 다른 사용자의 리소스 존재 여부를 감추기 위해 404를 반환한다.

Security Group은 VPC·Node·Database 경계, NetworkPolicy는 Pod 트래픽 분리, IAM/IRSA는 AWS API 권한을 담당한다. Pod는 runAsNonRoot, no privilege escalation, dropped capabilities, RuntimeDefault seccomp, no hostNetwork/PID/IPC/hostPath를 기준으로 한다.

7.3 Secrets and observability

Secrets Manager는 External Secrets Operator만 직접 읽고 Kubernetes Secret을 통해 Pod에 전달한다. 일반 애플리케이션이 Secrets Manager를 직접 읽는 것을 기준으로 두지 않으며 static AWS access key를 사용하지 않는다. 워크로드에는 IRSA를 적용한다.

Metric은 Prometheus/Grafana/Alertmanager, 운영 로그는 Fluent Bit와 CloudWatch Logs, 관리 감사는 AuditLog, 학습 실행 기록은 TrainingJobEvent로 남긴다. Prometheus label에는 고카디널리티 리소스 ID를 넣지 않는다. 동기 요청은 request_id, 비동기 흐름은 event_id + resource_id로 연계 추적한다. 분산 추적 백엔드는 초기 범위 밖이다.

7.4 Internal analytics and data access

내부 사용자는 공개 End User 경로와 분리된 Private Access Boundary를 통과한다. 접근 흐름은 Internal User → 승인된 VPN 또는 조직 내부 접근 경로 → MLflow / Athena이며, MLflow는 실험 추적을, Athena → Glue Data Catalog → S3 Parquet 경로는 대용량 Dataset과 InferenceLog 분석을 담당한다. 특정 VPN 제품이나 별도 인증 시스템은 추가하지 않고, 인증된 내부 주체를 기존 AWS IAM 역할과 Kubernetes RBAC 권한에 매핑해 최소 권한을 적용한다.

Role	허용된 접근	권한 경계
ML Engineer	MLflow, Athena, 학습 아티팩트	MLflow 실험 read/write, Training artifact read, Dataset metadata query
Data Analyst	Athena, Glue Catalog, S3 분석 데이터	Athena query, S3 Parquet read-only
Platform Operator	CloudWatch, Prometheus/Grafana	운영 관측과 장애 대응 권한

MLflow는 실험 결과, metric, hyperparameter, 실행 환경과 아티팩트 계보를 추적한다. MLflow는 Model Registry가 아니며, 모델 finalization과 deployment 상태의 SSOT는 RDS의 ModelVersion이다. Dataset과 추론 분석은 S3 Parquet, Glue Data Catalog, Athena로 수행해 RDS OLTP 워크로드와 분리한다.

Design Decision - S3, Glue and Athena for analytics
 Problem: 트랜잭션 상태 DB에 분석 스캔을 섞을 수 없다.
 Choice: 분석 데이터는 S3 Parquet와 Glue/Athena로 분리한다.
 Why: OLTP 성능과 분석 확장성을 분리한다.
 Trade-off: 데이터 최신성과 Catalog 운영은 별도 관리 대상이다.

8. Deployment Architecture & Implementation Scope

8.1 Runtime and delivery

운영 목표는 2 AZ, Public/Private Application/Private Data Subnet의 단일 VPC다. ALB는 Backend API와 Inference Gateway Pod만 ip target으로 연결한다. Managed Node Group은 상시 Component를, Karpenter GPU NodePool은 Training Job을 담당한다. EKS API는 Private와 restricted-public endpoint를 사용한다.

CI/CD는 GitHub Actions의 test, lint, build, vulnerability scan에서 시작해 ECR에 image를 저장하고, Helm desired state를 Argo CD가 EKS에 동기화하는 구조다. 이는 설계 범위이며 실제 CI/CD 구축 완료를 주장하지 않는다.

의도적으로 Redis, Kafka, EventBridge, KEDA, Knative, service mesh, distributed tracing backend, ONNX, GPU serving, Dask-CUDA, RAPIDS/cuML, canary/traffic split, stable alias, RDS Proxy, Read Replica, Multi-AZ DB Cluster를 초기 기준 구성에서 제외한다. 기능 부족이 아니라 현재 요구에 비해 운영 복잡도와 구현 비용의 기여가 낮기 때문이다.

8.2 Designed versus implemented

Scope	Status
EKS production architecture	Designed
S3 업로드 및 처리 흐름	Designed
SQS, Outbox와 Controller	Designed
GPU 학습과 Karpenter	Designed
MLflow, KServe, Glue/Athena	Designed
Security and observability baseline	Designed
RDB schema and migrations	Implemented for assignment
PostgreSQL local database	Implemented for assignment
Limited RDB-backed APIs	Implemented for assignment
Docker Compose evaluation	Implemented for assignment

초기 Codebase의 구현 경계는 modular monolith다. 논리적 Domain과 Process 책임은 분리하되, 각 운영 Component를 독립 Microservice로 모두 구현했다고 주장하지 않는다. 운영 요구가 구체화되면 DR/RTO/RPO, PITR/backup retention, cross-region recovery, durable InferenceLog delivery, 관측 데이터 retention/threshold/routing, MLflow Private 관리 연결, Secret 변경 시 재배포 책임자를 확정한다.

9. Conclusion

ArgMax Mini는 API가 대용량 파일 전송이나 장시간 GPU 실행을 직접 소유하지 않도록 설계한다. RDS PostgreSQL은 업무 상태와 실행 의도를 보존하고, S3는 변경 불가능한 데이터와 아티팩트를, SQS는 at-least-once wake-up signal을, Kubernetes Job과 KServe는 각각 데이터 처리·학습과 서빙 실행을 담당한다.

이 분리는 분산 시스템의 불일치를 제거한다고 주장하지 않는다. 대신 중복 전달(at-least-once delivery), 외부 호출의 불확실한 결과, Controller와 Kubernetes의 재시작을 전제로 멍등성과 Reconciliation으로 RDS SSOT에 기록된 상태에 수렴시킨다. 변경 불가능한 DatasetVersion과 ModelVersion, 명시적 배포 연결, Gateway/KServe 경계는 재현 가능한 계보와 안전한 운영 확장을 위한 최소한의 기반을 제공한다.